



MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 9

Optimasi Non-Linear: Gradient &

Abstrak

Pertemuan ini membahas Non-Linear Programming (NLP) untuk masalah rekayasa tanpa dan dengan kendala:

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 3.3 – Mampu menggunakan teknologi 4.0 untuk mengoptimalkan sistem otomatisasi

MODUL INTERAKTIF

<https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-9.html>

Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Linear Programming (Pertemuan 9) mengasumsikan fungsi objektif dan kendala bersifat linier, padahal banyak fenomena rekayasa (gaya drag proporsional v^2 , energi kinetik proporsional v^2) bersifat non-linier. Pertemuan kesepuluh ini membahas Non-Linear Programming (NLP): metode gradient-based untuk masalah tanpa kendala, serta KKT conditions dan penalty/barrier method untuk masalah berkendala.

Posisi Pertemuan 10 dalam Alur Mata Kuliah Optimalisasi & Otomasi

Gambar 1. Posisi Pertemuan 10 (Optimasi Non-Linear) dalam alur mata kuliah, di antara Linear Programming (P₉) dan Metaheuristik (P₁₁).

Gambar 1 menunjukkan posisi Pertemuan 10 dalam alur mata kuliah.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 3.3:** Mampu menggunakan teknologi 4.0 untuk mengoptimalkan sistem otomasi, yaitu menerapkan metode gradient-based dan constraint handling (`scipy.optimize`) sebagai teknologi komputasi 4.0 untuk mengoptimalkan parameter desain sistem otomasi non-linier (CPMK 3).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan syarat optimalitas orde-pertama dan orde-kedua; membedakan gradient descent, Newton's method, dan BFGS; menjelaskan KKT conditions; serta menerapkan ``scipy.optimize.minimize`` pada masalah desain berkendala. Hubungan ini dirangkum dalam Persamaan (1).

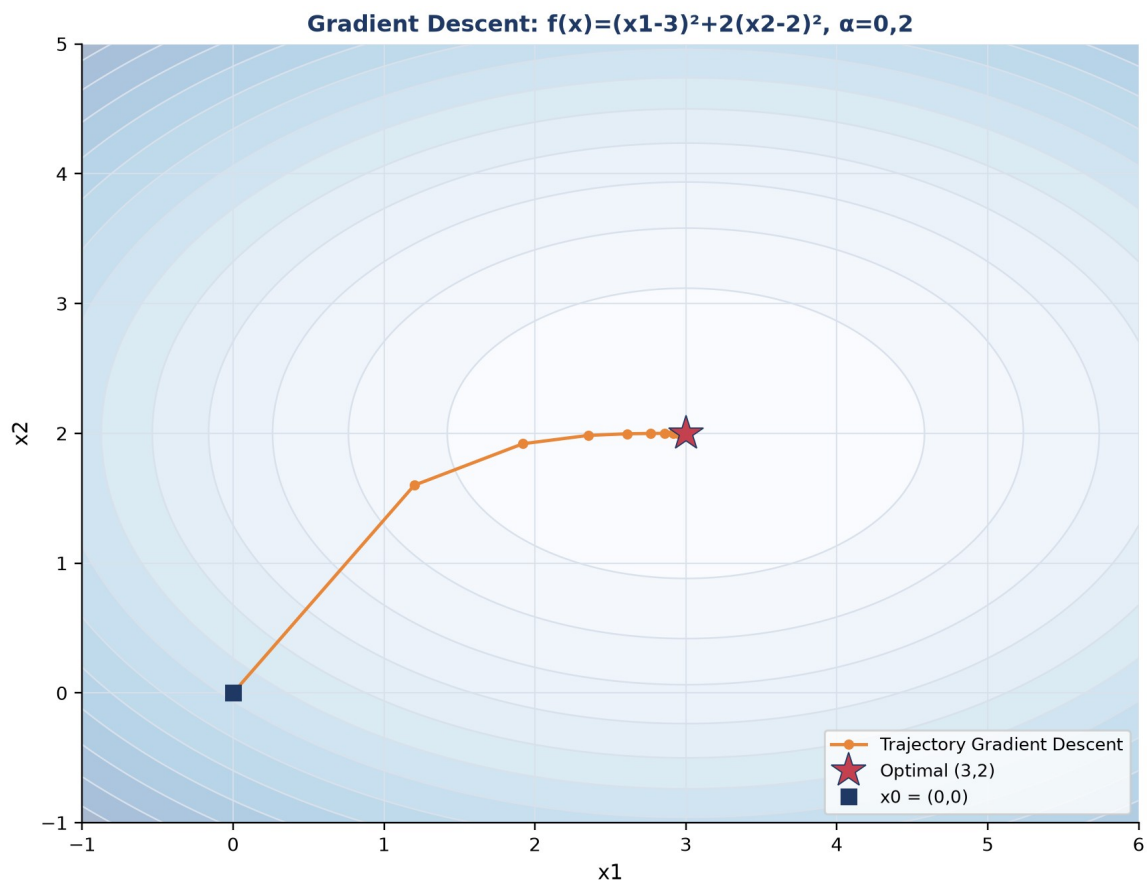
GRADIENT DESCENT & LINE SEARCH

x_0 (initial guess) $dk = -\nabla f(x_k)$ (direction) ak (step size) berhenti saat $\|\nabla f\| < \varepsilon$ (1)

Steepest Descent: Syarat orde-pertama untuk local minimum tanpa kendala adalah $\nabla f(x^*)=0$ (titik stasioner). Gradient descent bergerak dari x_0 mengikuti arah negative gradient: $x_{k+1}=x_k-\alpha \cdot \nabla f(x_k)$. Untuk $f(x_1,x_2)=x_1^2+2x_2^2$ dari $x_0=(3,2)$ dengan $\nabla f=(6,8)$ dan $\alpha=0,1$, iterasi pertama menghasilkan $x_1=(2,4; 1,2)$.

Backtracking Line Search: Backtracking line search (Armijo rule) memilih α secara adaptif: $f(x_k+\alpha d_k) \leq f(x_k)+c \cdot \alpha \cdot \nabla f^T d_k$ dengan $c \approx 10^{-4}$, lalu α dikurangi bertahap (faktor $p=0,5$) sampai kondisi terpenuhi. Ini mencegah overshoot tanpa harus mencoba-coba α secara manual.

Conjugate Gradient: Conjugate Gradient (Hestenes-Stiefel, 1952) memodifikasi arah pencarian $d_{k+1}=-\nabla f+\beta \cdot d_k$ sehingga untuk fungsi kuadratik konvergen dalam maksimal n iterasi, jauh lebih cepat dari steepest descent murni.



Gambar 2. Trajectory gradient descent pada kontur $f(x)=(x_1-3)^2+2(x_2-2)^2$ dari $x_0=(0,0)$ menuju optimal $(3,2)$, $\alpha=0,2$.

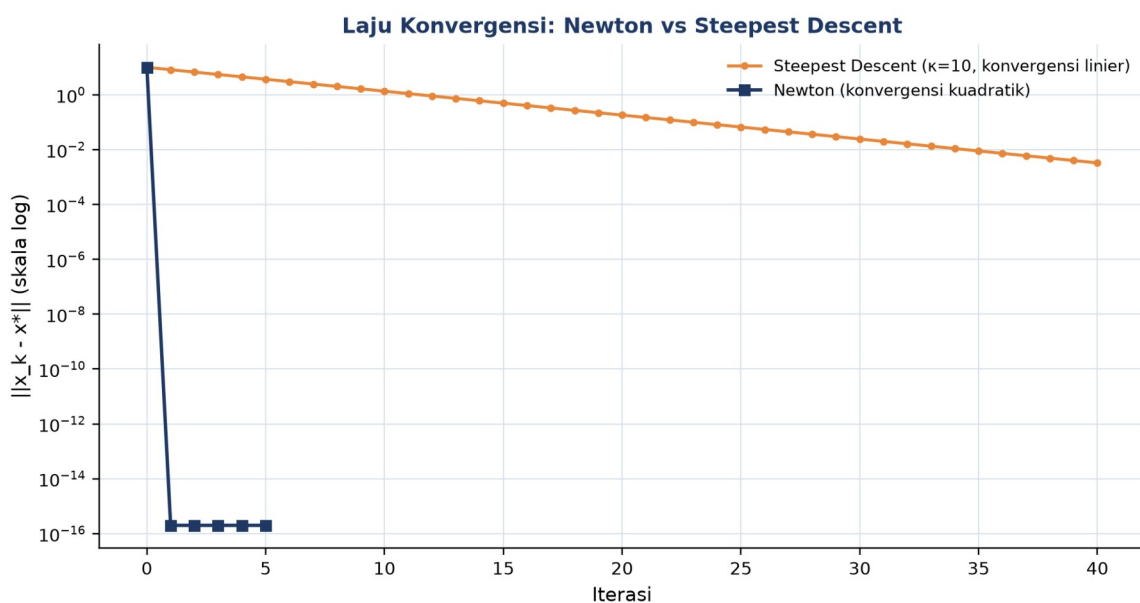
Gambar 2 mengilustrasikan trajectory gradient descent menuju titik optimal; arah pergerakan selalu tegak lurus terhadap garis kontur di tiap titik.

Contoh: Optimasi Tinggi Antena. Optimasi tinggi antena $f(h)=1000/h+50h^2$ untuk meminimalkan biaya kabel dan material tiang: $f'(h)=-1000/h^2+100h=0$ menghasilkan $h^*=\text{akar_pangkat_tiga}(10)\approx 2,154$ m; uji konveksitas $f''(h)=2000/h^3+100>0$ menegaskan ini adalah minimum global. Bentuk ringkasnya dituliskan pada Persamaan (2).

NEWTON'S METHOD & QUASI-NEWTON

Newton: $dk = -[\nabla^2 f]^{-1} \cdot \nabla f$ **BFGS update:** $B_{k+1} \approx B_k + \text{koreksi}(sk, yk)$ **L-BFGS:**
simpan m pasangan terakhir (2)

Pure Newton: Newton's method memakai informasi Hessian (turunan kedua) untuk membangun model kuadratik lokal, arah pencarian $d=-H^{-1} \cdot \nabla f$. Konvergensinya kuadratik: $\|x_{k+1}-x^*\| \leq C \cdot \|x_k-x^*\|^2$, jauh lebih cepat dari gradient descent yang hanya konvergen linier, tetapi butuh biaya $O(n^3)$ untuk invers Hessian tiap iterasi.



Gambar 3. Perbandingan laju konvergensi Newton (kuadratik) vs Steepest Descent (linier, condition number $\kappa=10$) pada fungsi kuadratik.

Gambar 3 membandingkan laju konvergensi kedua metode secara log-skala; Newton mencapai presisi tinggi hanya dalam beberapa iterasi, sedangkan steepest descent memerlukan puluhan iterasi terutama saat condition number tinggi.

- **BFGS:** BFGS (Broyden-Fletcher-Goldfarb-Shanno, 1970) mengaproksimasi Hessian hanya dari informasi gradient via secant condition $sk=x_{k+1}-x_k$, $yk=\nabla f_{k+1}-\nabla f_k$; konvergensi super-linier tanpa perlu menghitung Hessian eksak.

- **L-BFGS:** menyimpan hanya $m=5-20$ pasangan (s,y) terakhir sehingga kebutuhan memori $O(mn)$, cocok untuk masalah berdimensi besar.
- **Trust Region:** membatasi langkah d dalam radius kepercayaan $\|d\| \leq \Delta_k$; lebih robust dibanding line search murni untuk fungsi non-konveks.

Tabel 1. Perbandingan lima algoritma optimasi non-linear berdasarkan laju konvergensi, biaya per iterasi, dan kebutuhan turunan.

Algoritma	Konvergensi	Cost/Iterasi	Kebutuhan	Cocok Untuk
Steepest Descent	Linier	$O(n)$	Gradient saja	Konveks well-conditioned, sederhana
Conjugate Gradient	Super-linier (kasus kuadratik)	$O(n)$	Gradient saja	Sparse, skala besar, mirip kuadratik
Newton	Kuadratik	$O(n^3)$	Hessian eksak	n kecil, Hessian tersedia analitik
BFGS (Quasi-Newton)	Super-linier	$O(n^2)$	Gradient saja	Default unconstrained NLP, n hingga 1000
L-BFGS-B	Super-linier	$O(mn)$	Gradient, bounds OK	Skala besar, ML, identifikasi parameter

Tabel 1 merangkum trade-off lima algoritma optimasi non-linear yang umum dipakai.

Tip Praktis: Sebelum memakai gradient analitik dalam kode produksi, validasi dulu dengan ``scipy.optimize.check_grad(f, jac, x0)`` untuk memastikan turunan yang diimplementasikan benar. Persamaan (3) memadatkan aturan tersebut.

KKT CONDITIONS & CONSTRAINT HANDLING

$$\begin{aligned} \text{Stationarity: } \nabla f + \sum \lambda_i \nabla h_i + \sum \mu_j \nabla g_j &= 0 & \text{Primal feasibility: } g(x) \leq 0, h(x) &= 0 & \text{Dual} \\ \text{feasibility: } \mu_j \geq 0 & & \text{Complementary slackness: } \mu_j \cdot g_j(x) &= 0 \end{aligned} \quad (3)$$

- **Lagrangian Function:** $L(x, \lambda) = f(x) + \lambda^T \cdot h(x) + \mu^T \cdot g(x)$; turunan Lagrangian terhadap x memberi syarat stasioner, dan $\partial f^* / \partial b = -\lambda^*$ menghubungkan multiplier dengan sensitivitas terhadap RHS kendala.
- **Complementary Slackness:** $\mu_j \cdot g_j(x^*) = 0$ berarti kendala aktif ($g=0$) boleh punya $\mu > 0$, sedangkan kendala tidak aktif ($g < 0$) harus punya $\mu = 0$.
- **Penalty Method:** mengubah masalah berkendala menjadi tanpa kendala dengan menambah term $p \cdot \sum \max(0, g_j(x))^2$ ke objektif; saat p menuju tak hingga, solusi mendekati solusi constrained sesungguhnya, tetapi p besar menyebabkan ill-conditioning.

- **Barrier Method:** menambah term $-\mu \cdot \sum \log(-g_j(x))$ yang menuju tak hingga saat mendekati batas kendala, menjaga iterasi selalu berada di interior; mendasari interior-point method (Karmarkar, 1984).
- **Sequential Quadratic Programming (SQP):** menyelesaikan serangkaian sub-masalah quadratic programming yang mengaproksimasi NLP asli secara lokal; efisien untuk kendala non-linier umum.
- **Augmented Lagrangian:** $L_A = f + \lambda^T h + (\rho/2) \|h\|^2$; menggabungkan penalty dan multiplier Lagrange (Powell 1969, Hestenes 1969) sehingga ρ tidak perlu sangat besar untuk akurasi tinggi.

Tabel 2. Perbandingan lima pendekatan constraint handling di `scipy.optimize` berdasarkan tipe kendala yang didukung dan robustness.

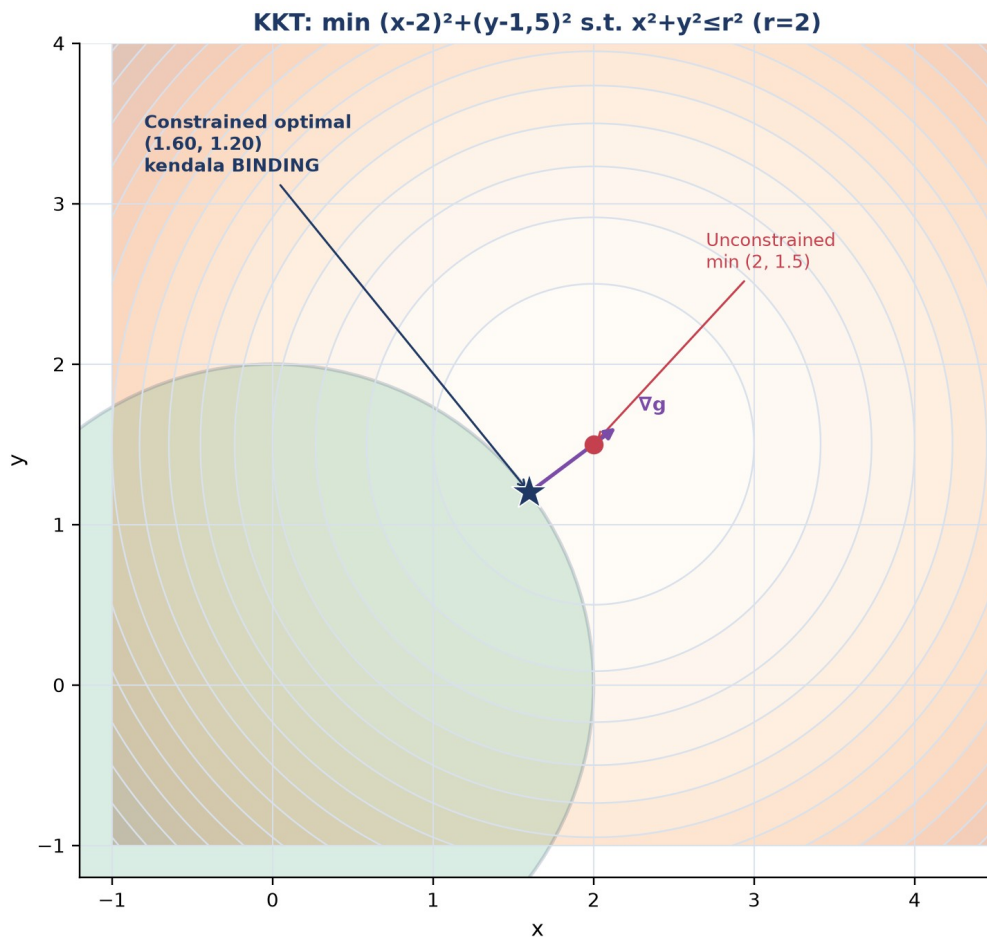
Method	Tipe Kendala	Robustness	Cocok Untuk
L-BFGS-B	Hanya bounds	Tinggi	Bounds saja, scalable
SLSQP	Equality, inequality, bounds	Tinggi	Default NLP berkendala, $n < 100$
trust-constr	Equality, inequality, bounds, sparse	Sangat tinggi	Modern, robust, skala besar
COBYLA	Hanya inequality (tanpa equality)	Sedang	Derivative-free, kendala $\leq$ saja
Penalty manual	Apapun	Rendah (ill-conditioning)	Belajar konsep, prototyping cepat

Tabel 2 membandingkan lima metode constraint handling yang tersedia di `scipy.optimize` beserta karakteristik kecocokannya.

Heuristik Pemilihan Solver dalam Praktik Rekayasa: Pemilihan solver dari Tabel 2 dalam praktik rekayasa sering mengikuti heuristik sederhana: mulai dengan SLSQP sebagai default karena dukungannya yang luas terhadap tipe kendala dan kecepatan konvergensi yang baik pada kebanyakan masalah desain berdimensi sedang ($n < 100$), lalu beralih ke trust-constr bila SLSQP gagal konvergen atau masalah melibatkan kendala sparse berskala besar. COBYLA dicadangkan khusus untuk kasus di mana turunan fungsi objektif tidak tersedia secara analitik maupun numerik (misalnya objektif berasal dari simulasi black-box seperti FEM atau CFD), sementara penalty method manual umumnya hanya dipakai untuk tujuan pembelajaran konsep, bukan produksi, karena rawan ill-conditioning saat parameter ρ perlu dibesarkan untuk mencapai akurasi tinggi.

Interpretasi Geometris Kondisi KKT: Interpretasi geometris KKT conditions menjadi jauh lebih intuitif saat divisualisasikan: pada titik optimal constrained, gradient fungsi objektif ∇f harus berupa kombinasi linear dari gradient kendala aktif ∇g (dengan koefisien non-negatif untuk kendala inequality), yang secara geometris berarti tidak ada arah pergerakan yang

SEKALIGUS meningkatkan feasibilitas dan memperbaiki nilai objektif. Bila kondisi ini gagal terpenuhi pada suatu titik, maka titik tersebut BUKAN optimal, karena selalu ada arah perbaikan (feasible descent direction) yang belum dieksplorasi oleh solver.



Gambar 4. Visualisasi KKT: $\min (x-2)^2+(y-1,5)^2$ dengan kendala $x^2+y^2 \leq 4$; optimal tanpa kendala (2; 1,5) berada di luar daerah feasibel, sehingga solusi constrained bergeser ke batas lingkaran dengan kendala BINDING.

Gambar 4 menunjukkan kasus di mana optimum tanpa kendala berada di luar daerah feasibel; solusi constrained bergeser ke titik pada batas lingkaran di mana gradient kendala ∇g sejajar dengan arah menuju optimum tanpa kendala, sesuai kondisi stasioner KKT.

Contoh: Tangki Silinder Volume Tetap. Untuk desain tangki silinder dengan volume tetap V_0 , minimisasi luas permukaan $f=2\pi R^2+2\pi RH$ dengan kendala $\pi R^2H=V_0$ menghasilkan solusi analitik $H=2R$, $R^*=\text{akar_pangkat_tiga}(V_0/2\pi)\approx 0,542$ m dan $H\approx 1,084$ m (untuk $V_0=1$ m³).

Peringatan, Trap KKT yang Sering Diabaikan: Empat jebakan KKT yang sering diabaikan: constraint qualification yang tidak terpenuhi membuat KKT tidak valid; untuk NLP non-konveks, KKT hanya syarat perlu bukan cukup (bisa jadi saddle point); penalty dengan

p terlalu besar menyebabkan ill-conditioning numerik; serta lupa memeriksa ``result.constr_violation`` untuk memastikan solusi benar-benar feasible.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Mendaki Gunung di Kabut Tebal:** mendaki gunung dalam kabut tebal, hanya bisa merasakan kemiringan tanah di bawah kaki dan melangkah ke arah yang paling menurun/menaik, persis seperti gradient descent yang hanya memakai informasi lokal.
- **Naik Sepeda & Local Stability:** pengendara sepeda terus melakukan koreksi kecil untuk menjaga keseimbangan (feedback iteratif); sepeda fixed-gear di tanjakan curam adalah analogi jebakan non-konveks (local trap).
- **Audio Compressor & Knee Curve:** mastering engineer mengatur threshold, ratio, attack, dan release sehingga loudness tetap di atas -14 LUFS, sebuah optimasi berkendala interaktif.
- **Cruise Control Modern (MPC):** cruise control modern (MPC, dipakai Tesla Autopilot dan Mercedes EQS) menyelesaikan NLP berkendala setiap 100 milidetik untuk menjaga jarak aman sambil meminimalkan konsumsi bahan bakar.
- **Training Neural Network (Deep Learning):** model bahasa besar seperti GPT-4 dan AlphaFold dilatih via SGD/Adam/AdamW pada landscape loss yang sangat non-konveks berdimensi miliaran.
- **Camera Autofocus (Phase & Contrast Detection):** autofocus kamera memakai hill-climbing berbasis gradient finite-difference dari nilai kontras/fase untuk menemukan posisi lensa optimal.

Pola Umum: Pola yang sama di semua analogi: keputusan diambil berulang berdasarkan informasi lokal (slope/gradient) tanpa memerlukan gambaran global sistem, dan iterasi terus dilakukan hingga tidak ada perbaikan berarti lagi.

APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

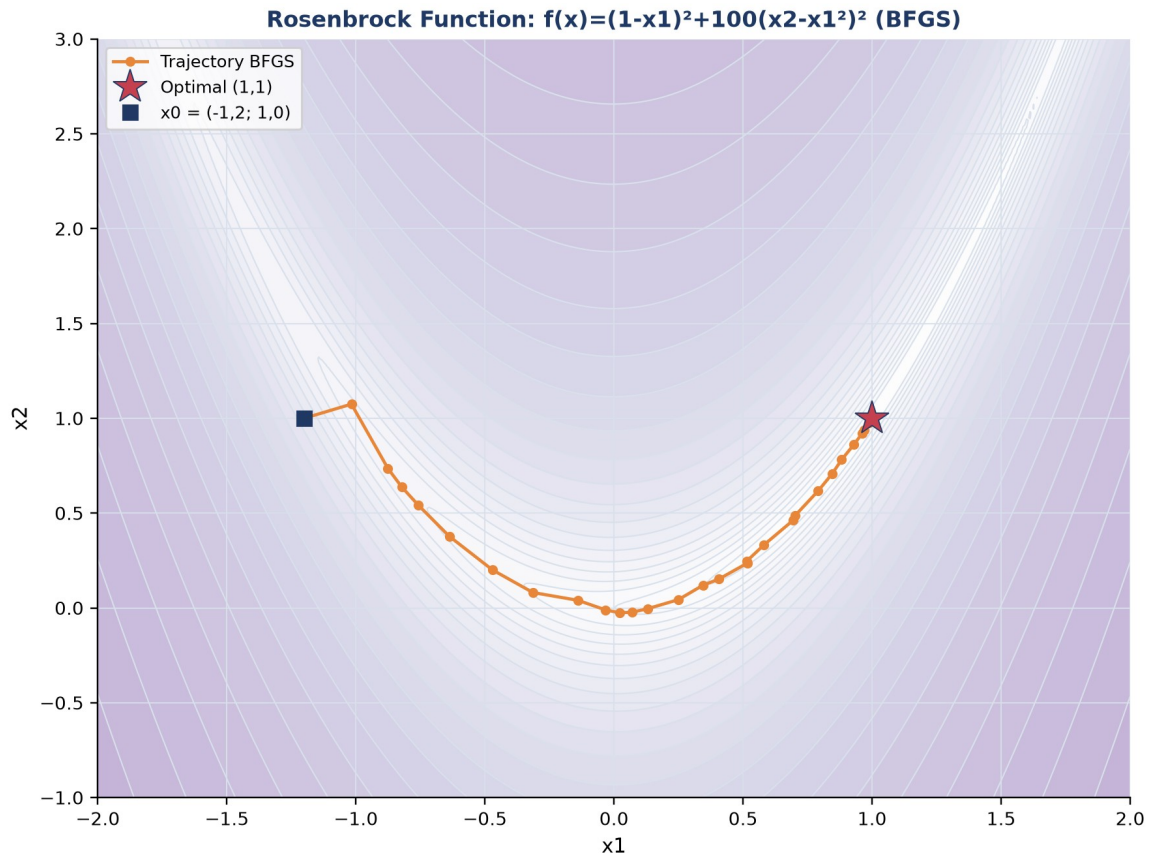
- **Design Parameter Optimization:** desain parameter pegas coil suspensi otomotif (diameter kawat d , jumlah lilitan N , panjang L) dioptimasi untuk minimisasi massa dengan kendala tegangan luluh, dipakai Mercedes, BMW, dan Toyota.
- **PID/MPC Controller Tuning:** tuning parameter PID/MPC untuk minimisasi ITAE/ISE, diimplementasikan dengan library ``do-mpc`` dan ``CasADi``.

- **Topology Optimization (FEM):** topology optimization berbasis FEM (metode SIMP) dipakai Boeing 787 untuk bracket, BMW iX untuk gearbox, dan Tesla untuk giga-casting demi minimisasi berat struktur.
- **Robot Trajectory & Optimal Control:** perencanaan trajectory robot industri (KUKA, Universal Robots) meminimalkan waktu tempuh dengan kendala dinamika dan batas sendi.

Peringatan, Jebakan Praktis Formulasi NLP: Enam jebakan praktis formulasi NLP: lupa memvalidasi gradient analitik sebelum dipakai solver; memilih titik awal yang jauh dari solusi pada masalah non-konveks; mengabaikan scaling variabel yang berbeda orde besaran; menganggap hasil SLSQP tunggal sudah global optimal; lupa memeriksa constraint violation pada solusi akhir; serta tidak melakukan multi-start pada masalah dengan banyak local minimum.

IMPLEMENTASI PYTHON NLP DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini, berpuncak pada studi kasus desain pegas coil suspensi. Lingkungan: conda env optoauto dengan paket numpy, scipy, matplotlib, pandas, sympy; notebook p10_nonlinear_programming.ipynb.



Gambar 5. Trajectory BFGS pada Rosenbrock function $f(x)=(1-x_1)^2+100(x_2-x_1^2)^2$, benchmark klasik NLP dengan lembah sempit melengkung.

Gambar 5 menunjukkan trajectory BFGS pada fungsi Rosenbrock, benchmark klasik yang menguji kemampuan solver menavigasi lembah sempit melengkung menuju optimal (1,1).

- **Cell 1:** gradient descent manual dan `scipy.optimize.minimize(method='BFGS')` untuk min $f(x,y)=(x-3)^2+2(y-2)^2$ (solusi analitik $x^*=3$, $y^*=2$, $f^*=0$); plot kontur dan trajectory.
- **Cell 2:** constrained NLP desain pegas coil: minimisasi massa dengan kendala tegangan geser dan defleksi, memakai `method='SLSQP'`, konstanta $p=7,85e-6$ kg/mm³, $G=79300$ MPa, $\tau_y=600$ MPa, $F=500$ N, $D=50$ mm, $\delta_{target}=25$ mm, bounds $d \in [2,15]$, $N \in [5,30]$, $L \in [50,200]$, $x_0=[8,12,120]$.
- **Cell 3:** KKT dan Lagrangian tangki silinder volume tetap ($V_0=1,0$ m³): solusi simbolik via `sympy` diverifikasi numerik dengan `scipy.minimize(method='SLSQP')`.
- **Cell 4:** fungsi `solve_design_nlp()` reusable dengan validasi gradient (`check_grad`) dan multi-start (`np.random.seed(42)`) untuk masalah non-konveks; didemonstrasikan ulang pada masalah pegas Cell 2 dengan `n_start=10`.

Validasi Solusi via Multi-Start: Cell 4 menegaskan pentingnya validasi solusi NLP secara sistematis sebelum dipakai sebagai keputusan desain final: fungsi `solve_design_nlp()` tidak hanya menjalankan solver sekali, tetapi mengulangnya dari sepuluh titik awal acak

berbeda (multi-start) dan membandingkan hasil `result.fun` dari tiap run. Bila seluruh sepuluh run konvergen ke nilai objektif yang sama (dalam toleransi numerik kecil), keyakinan bahwa solusi tersebut adalah optimum global (bukan sekadar local minimum) menjadi jauh lebih tinggi, meski tetap tidak ada jaminan matematis mutlak untuk masalah non-konveks umum.

```
p10_nonlinear_programming.ipynb — Cell 2
from scipy.optimize import minimize

def spring_mass(x):
    d, N, L = x
    return rho*np.pi*(d**2/4)*(np.pi*D)*N

cons = [{'type':'ineq', 'fun': stress_con},
        {'type':'ineq', 'fun': deflect_con}]
bounds = [(2,15), (5,30), (50,200)]

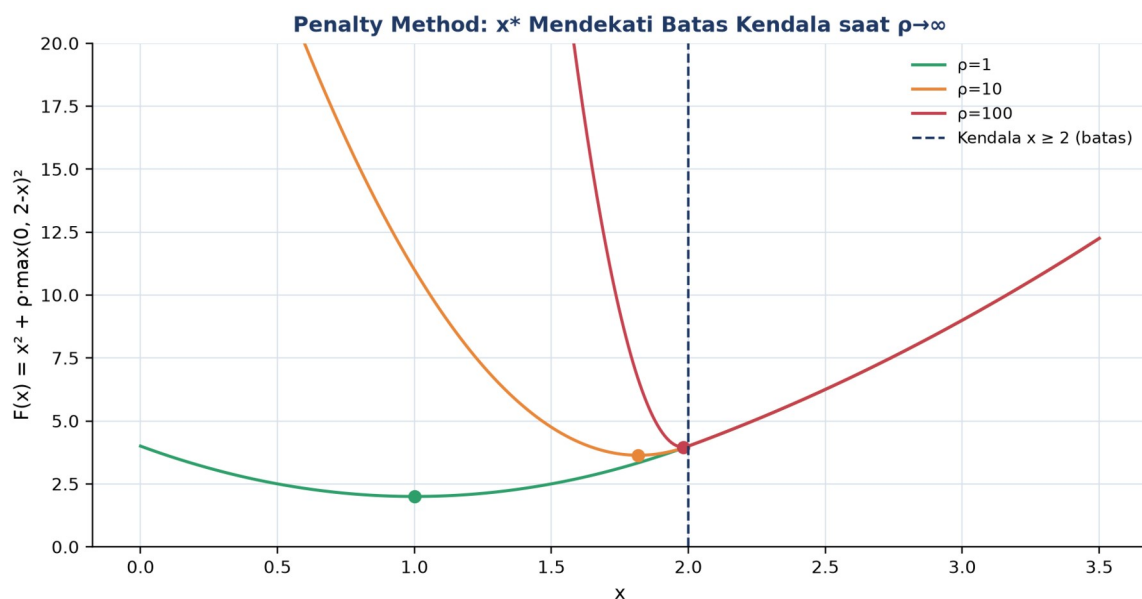
res = minimize(spring_mass, x0=[8,12,120],
               method='SLSQP', bounds=bounds, constraints=cons)

print(f'massa optimal = {res.fun*1000:.1f} gram')
```

Gambar 6. Potongan kode Cell 2: formulasi dan penyelesaian desain pegas coil dengan `scipy.optimize.minimize` method SLSQP.

Gambar 6 menunjukkan potongan kode inti Cell 2.

Penalty Method dalam Praktik: Untuk masalah non-konveks, satu kali menjalankan SLSQP dari satu titik awal berisiko terjebak local minimum. Multi-start (mencoba banyak titik awal acak dan mengambil hasil terbaik) atau metaheuristik (GA/PSO, dibahas Pertemuan 11) lebih realistis untuk mendekati solusi global.



Gambar 7. Penalty method: $F(x)=x^2+p \cdot \max(0,2-x)^2$ untuk $p=1, 10, 100$; solusi x^* semakin mendekati batas kendala $x=2$ saat p membesar.

Gambar 7 menunjukkan bagaimana penalty method secara bertahap mendorong solusi menuju batas kendala saat parameter penalty p diperbesar.

TUGAS PERTEMUAN 10

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 10



Gambar 8. Distribusi 50 poin Tugas Pertemuan 10 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal $\times$ 1 poin)

1. Pernyataan paling tepat tentang syarat orde-pertama (necessary condition) bagi local minimum fungsi $f(x)$ tanpa kendala adalah...

- A. $f(x^*) = 0$ (nilai fungsi nol)
- B. $\nabla f(x^*) = 0$ (gradient nol, disebut titik stasioner)
- C. $\nabla^2 f(x^*) = 0$ (Hessian nol)
- D. $x^* = 0$ (variabel keputusan nol)

2. Pada gradient descent dengan formula $x_{k+1} = x_k - \alpha \cdot \nabla f(x_k)$, peran step size α adalah...

- A. Menentukan jumlah dimensi variabel
- B. Mengontrol seberapa jauh melangkah ke arah negative gradient; terlalu besar overshoot/divergen, terlalu kecil konvergen lambat
- C. Memberi penalty terhadap kendala
- D. Selalu harus diset $\alpha = 1,0$ saja

3. Bedanya Newton's method dari steepest descent adalah...

- A. Newton tidak butuh gradient, hanya pakai nilai fungsi

- **B.** Newton pakai informasi Hessian (turunan kedua) untuk model lokal kuadratik, arah $d = -H^{-1} \cdot \nabla f$; konvergensi kuadratik (jauh lebih cepat dari linier)
- **C.** Newton hanya untuk fungsi linier; steepest descent untuk non-linier
- **D.** Newton tidak menjamin konvergensi sama sekali

4. Method 'BFGS' di `scipy.optimize.minimize` termasuk dalam keluarga...

- **A.** Gradient-free derivative-free method (seperti Nelder-Mead)
- **B.** Quasi-Newton, aproksimasi Hessian dari gradient saja, konvergensi super-linier, default `scipy` untuk unconstrained NLP
- **C.** Gradient-only method seperti steepest descent murni
- **D.** Linear programming solver simpleks variant

5. Untuk masalah NLP berkendala $\min f(x)$ s.t. $g(x) \leq 0$, $h(x)=0$, Lagrangian didefinisikan sebagai...

- **A.** $L = f(x) - g(x) - h(x)$
- **B.** $L(x, \lambda, \mu) = f(x) + \lambda^T \cdot h(x) + \mu^T \cdot g(x)$ dengan $\mu \geq 0$ untuk inequality
- **C.** $L = f(x) \cdot g(x) \cdot h(x)$ (perkalian)
- **D.** $L = \nabla f(x)$ saja, tanpa multiplier

6. Complementary slackness dalam KKT conditions menyatakan untuk tiap kendala $g_j(x) \leq 0$ dengan multiplier $\mu_j \geq 0$...

- **A.** $\mu_j + g_j(x) = 0$ selalu
- **B.** $\mu_j \cdot g_j(x^*) = 0$; kendala aktif ($g=0$) bisa punya $\mu > 0$, kendala tidak aktif ($g < 0$) harus $\mu = 0$
- **C.** $\mu_j = g_j(x)$ selalu sama nilainya
- **D.** $\mu_j > g_j(x)$ tanpa kondisi tambahan

7. Penalty method mengkonversi NLP berkendala menjadi unconstrained dengan cara...

- **A.** Menghapus semua kendala dari masalah
- **B.** Menambah term $p \cdot \sum \max(0, g_j)^2$ ke objektif; saat p menuju tak hingga, solusi unconstrained menuju solusi constrained
- **C.** Mengubah variabel x menjadi $\log(x)$
- **D.** Hanya untuk LP, tidak berlaku untuk NLP

8. Untuk NLP berkendala umum (equality dan inequality) di `scipy`, method yang paling tepat dari `minimize` adalah...

- **A.** `method='BFGS'` (hanya unconstrained)
- **B.** `method='SLSQP'` atau `'trust-constr'`, mendukung equality, inequality, dan bounds
- **C.** `method='Nelder-Mead'` (derivative-free, tidak handle constraints langsung)
- **D.** `method='CG'` (conjugate gradient, unconstrained)

9. Untuk fungsi konveks, sembarang local minimum juga merupakan...

- A. Saddle point
- B. Global minimum, sifat fundamental fungsi konveks (Hessian semi-definit positif)
- C. Maksimum lokal di sisi lain
- D. Tidak ada hubungan sama sekali dengan global

10. Untuk masalah NLP non-konveks (banyak local minimum), strategi paling realistis untuk mendapat solusi mendekati global adalah...

- A. Pakai BFGS sekali saja dari titik default, pasti dapat global
- B. Multi-start dengan banyak titik awal random lalu ambil hasil terbaik, atau pakai metaheuristik (GA/PSO, Pertemuan 11)
- C. Ubah kendala secara acak hingga masalah jadi linier
- D. Tidak perlu metode khusus, non-konveks selalu solvable dengan gradient descent biasa

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan: gunakan `scipy.optimize.minimize`. Method 'BFGS' untuk unconstrained, 'SLSQP' untuk berkendala. Sediakan gradient analitik via `'jac=...'` bila bisa. Pastikan cek `result.success`.

C1. Selesaikan unconstrained NLP $\min f(x) = (x-4)^2$ dengan `scipy.optimize.minimize` method BFGS, $x_0=0$. Print nilai x optimal.

-  **HINT:** Definisikan fungsi tujuan sebagai λ , panggil `minimize(..., method='BFGS')`, lalu ambil x optimal dari `res.x`. Cek analitik: turunan nol di $x=4$.

C2. Untuk fungsi quadratic 2D $f(x,y)=(x-3)^2+2(y-2)^2$, hitung nilai gradient di titik (1,1) sebagai vektor norma: $\|\nabla f(1,1)\|=\sqrt{(\nabla f_x^2+\nabla f_y^2)}$. Print nilainya.

-  **HINT:** Gradient $\nabla f = (2(x-3), 4(y-2))$; evaluasi di titik yang diminta lalu hitung norma $= \sqrt{(\nabla f_x^2+\nabla f_y^2)}$ dengan `np.sqrt`.

C3. Selesaikan $\min f(x,y)=x^2+2y^2$ dengan `scipy.minimize` method BFGS, $x_0=[3,2]$. Print nilai f optimal ($f(x^*)$).

-  **HINT:** Panggil `minimize(..., method='BFGS')` dengan fungsi tujuan sebagai λ ; nilai f optimal dibaca dari `res.fun` (bisa sangat kecil karena floating point).

C4. Selesaikan NLP berkendala $\min f(x,y)=x^2+y^2$ s.t. $x+y=4$. Pakai method='SLSQP' dengan `constraints={'type':'eq', 'fun': lambda x: x[0]+x[1]-4}`. Print x optimal ($x[0]$).

-  **HINT:** Pakai method='SLSQP' dengan kendala equality lewat dict `{'type':'eq', 'fun': ...}`; mulai dari x_0 feasible dan baca x optimal dari `res.x`.

C5. Untuk masalah sebelumnya ($\min x^2+y^2$ s.t. $x+y=4$), hitung multiplier λ dari Lagrangian $L=x^2+y^2+\lambda(x+y-4)$. Solusi: $\partial L/\partial x=2x+\lambda=0$, $\partial L/\partial y=2y+\lambda=0$. Print nilai λ .

- **HINT:** Dari kondisi stasioner Lagrangian $\partial L/\partial x = 2x + \lambda = 0$, sehingga $\lambda = -2x$ dievaluasi di solusi optimal.

C6. Lakukan 1 iterasi gradient descent untuk $f(x) = x^2$ mulai dari $x_0 = 5$ dengan step size $\alpha = 0,3$. Formula: $x_1 = x_0 - \alpha \cdot f'(x_0)$. Print x_1 .

- **HINT:** Turunan $f'(x) = 2x$; terapkan rumus update $x_1 = x_0 - \alpha \cdot f'(x_0)$ dengan nilai x_0 dan α dari soal.

C7. Untuk fungsi $f(x) = x^4 - 4x^2 + 5$, hitung 1 step Newton's method mulai dari $x_0 = 3$ dengan formula $x_1 = x_0 - f'(x_0)/f''(x_0)$. Print x_1 .

- **HINT:** Turunkan $f'(x) = 4x^3 - 8x$ dan $f''(x) = 12x^2 - 8$, evaluasi di x_0 , lalu terapkan update Newton $x_1 = x_0 - f'(x_0)/f''(x_0)$.

C8. Validasi konveksitas $f(x) = x^4 + 2x^2 + 1$ di $x = 1$ via Hessian (turunan kedua). Hitung $f''(1)$. Print nilainya. Jika $f''(x) \geq 0$ di seluruh domain maka konveks.

- **HINT:** Hessian 1D = turunan kedua $f''(x) = 12x^2 + 4$; evaluasi di x yang diminta. Karena selalu lebih besar dari 0, fungsi bersifat strictly convex.

C9. Selesaikan NLP berkendala dengan bounds: $\min f(x,y) = (x-2)^2 + (y-3)^2$ s.t. $0 \leq x \leq 1, 0 \leq y \leq 5$. Pakai method='L-BFGS-B', $x_0 = [0.5, 0.5]$. Print x optimal.

- **HINT:** Pakai method='L-BFGS-B' dengan parameter bounds; jika optimum tak berkendala di luar batas, solver memproyeksikannya ke boundary. Baca x optimal dari $res.x$.

C10. Penalty method: untuk $\min f(x) = x^2$ s.t. $x \geq 2$, formulasi penalty $F(x) = x^2 + p \cdot \max(0, 2-x)^2$. Hitung $F(1.5)$ dengan $p = 100$. Print nilainya.

- **HINT:** Substitusikan x ke $F(x) = x^2 + p \cdot \max(0, 2-x)^2$; suku penalty hanya aktif saat kendala dilanggar. Saat p menuju tak hingga, solusi mendekati constraint binding ($x = 2$).

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong + submit = 0 poin dan tidak terkunci (bisa retry).

C11 (Hard). Rosenbrock Function, benchmark NLP klasik: $f(x,y) = (1-x)^2 + 100(y-x^2)^2$. Selesaikan dengan `scipy.minimize` method BFGS, $x_0 = [-1.2, 1.0]$. Print $x^* + y^*$ (jumlah dari koordinat minimum yang ditemukan optimizer).

- **HINT:** Rosenbrock ("banana function") punya solusi unik di $(1,1)$; selesaikan dengan `minimize(..., method='BFGS')` lalu jumlahkan komponen `res.x` dengan `.sum()`.

C12 (Hard). Tangki Silinder Volume Tetap: cari R, H untuk min surface area $f = 2\pi R^2 + 2\pi RH$ s.t. volume $\pi R^2 H = 1,0 \text{ m}^3$. Pakai method=SLSQP. Print R^* (radius optimal, m).

-  **HINT:** Pakai method='SLSQP' dengan kendala volume sebagai dict equality {'type':'eq', 'fun': lambda x: pi*R**2*H - V}; mulai dari x0 feasible dan baca R dari res.x. Solusi analitik optimal: $H=2R$.

C13 (Hard). Penalty Method Convergence: untuk $\min f(x)=x^2$ s.t. $x \geq 2$, formulasi penalty $F(x)=x^2+p \cdot \max(0, 2-x)^2$. Solve untuk $p=1, 10, 100$ dengan `scipy.minimize`. Print x^* untuk $p=100$.

-  **HINT:** Untuk tiap p , minimkan $F(x)=x^2+p \cdot \max(0, 2-x)^2$ dengan `minimize` lalu baca `res.x`; makin besar p , x^* makin mendekati constraint binding $x=2$.

C14 (Hard). Multi-Start untuk Non-Konveks: untuk $f(x)=x^4-8x^2+5$ (dua minimum di $x=\pm 2$, satu lokal di $x=0$), jalankan 10 kali `minimize BFGS` dari x_0 random uniform di $[-5,5]$ (seed=42). Print nilai f minimum global yang ditemukan.

-  **HINT:** Jalankan `minimize` berkali-kali dari x_0 acak (set seed), kumpulkan tiap `res.fun`, lalu ambil yang terkecil dengan `min` untuk menemukan minimum global.

C15 (Hard). Pegas Coil Engineering Design: $\min$ mass $m=\rho \cdot \pi \cdot (d^2/4) \cdot (\pi D) \cdot N$ dengan $\rho=7,85e-6$ kg/mm³, $D=50$ mm, $F=500$ N. Kendala: $\tau_{\max}=8FD/(\pi d^3) \leq 600$ MPa, $\delta=8FD^3N/(Gd^4) \geq 25$ mm ($G=79300$ MPa), $L \geq N \cdot d$. Bounds: $d \in [2,15]$, $N \in [5,30]$, $L \in [50,200]$. Pakai SLSQP. Print massa minimum (gram, $x_0=[8,12,120]$).

-  **HINT:** Formulasikan fungsi massa dan tiga kendala (tegangan, defleksi, panjang) sebagai dict inequality, lalu selesaikan dengan method='SLSQP' memakai bounds dan x_0 dari soal; massa dalam gram = `res.fun` x 1000.

DAFTAR PUSTAKA

- Usha, R. (2022). A mathematical model for nonlinear optimization which attempts membership functions to address the uncertainties. *Mathematics*, 10(10), 1743. <https://doi.org/10.3390/math10101743>
- Oancea, G. (2022). Machining parameters optimization based on objective function linearization. *Mathematics*, 10(5), 803. <https://doi.org/10.3390/math10050803>
- Bayá, G., Canale, E., Nesmachnow, S., De Sousa, A., & Sartor, P. (2022). Production optimization in a grain facility through mixed-integer linear programming. *Applied Sciences*, 12(16), 8212. <https://doi.org/10.3390/app12168212>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng.*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>

Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972.

<https://doi.org/10.3390/app12030972>

Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>